

Betaflight 커스텀 펌웨어

CLI 파라미터 설명서 (v2 + v3)

Alt Hold / Position Hold 하강 스틱 구간 확장 + EZ Disarm airmode 연동

항목	내용
기준 소스	betaflight/betaflight tags/2025.12.5
관련 이슈	GitHub betaflight/betaflight #15775 (본인 제출)
브랜치	custom-patch/alt-hold-throttle-range
대상 MCU	STM32F405 / STM32F7X2 / STM32H743
대상 기체	MARIO5, Mark4_6in, AOS_UL7_O4, TJRC_10, 8IN-KOPIS_X8, CHIMERA7, AOS_UL7_X8, Explorer LR4, X8_5INCH (총 9대, 3종 MCU)
작성일	2026-09-29

이 문서는 커스텀 빌드에 새로 추가된 CLI 파라미터 4종(v1/v2 3종 + v3 1종)의 사용법과, 그 파라미터들과 직접 연관되어 함께 알아야 할 기존(공식) Betaflight CLI 파라미터를 정리한 기술 설명서입니다. 커스텀 빌드는 Betaflight 공식 지원 대상이 아니며, 사용자 자체 책임 하에 시험·적용합니다.

목차

1. 개요 및 배경
2. 신규 커스텀 CLI 파라미터 (v1+v2+v3)
 - 2.1 alt_hold_full_low_is_max_descend
 - 2.2 alt_hold_deadband_low
 - 2.3 alt_hold_hover_throttle
 - 2.4 landing_disarm_airmode_off_only (v3)
3. 관련 기존(공식) CLI 파라미터
 - 3.1 alt_hold_deadband / alt_hold_climb_rate
 - 3.2 ap_hover_throttle
 - 3.3 min_check / max_check
 - 3.4 landing_disarm_threshold (EZ Disarm)
 - 3.5 mixer_type = EZLANDING 및 ez_landing_* 계열
 - 3.6 feature AIRMODE / aux BOXAIRMODE
4. 계산 공식 요약
5. 권장 설정 및 CLI 예시
6. 안전 점검 요약
7. 기체별 빌드/펌웨어 매핑

1. 개요 및 배경

Alt Hold(고도유지)/Position Hold(위치유지) 비행 중 스로틀 스틱으로 하강을 조작할 때, 하강 방향의 유효 스틱 구간이 상승 방향보다 지나치게 좁아 정밀한 하강 제어가 어렵다는 문제를 GitHub 이슈 #15775로 제출했습니다. 공식 반영까지 시간이 걸릴 수 있어, 우선 자체 패치 펌웨어를 빌드해 시험 적용하기로 했습니다. 이후 실비행 안전성 검토 과정에서 EZ Disarm(착지 충격 자동 디스암) 기능이 비행 중에도 airmode 상태와 무관하게 작동해 오작동 위험이 있다는 점이 추가로 확인되어, v3에서 이를 개선하는 토글 파라미터를 추가했습니다.

패치는 세 단계(v1, v2, v3)로 누적 적용되었으며, 모두 공식 소스코드 대비 opt-in(기본값 OFF 또는 기존값 상속) 방식이라 값을 바꾸지 않는 한 기존 공식 Betaflight와 동일하게 동작합니다.

버전	추가 내용	PG 버전 변경
v1	alt_hold_full_low_is_max_descend, alt_hold_deadband_low	PG_ALTHOLD_CONFIG 4→5
v2	alt_hold_hover_throttle (Alt Hold/Position Hold 전용 호버 스로틀)	PG_ALTHOLD_CONFIG 5→6
v3	landing_disarm_airmode_off_only (EZ Disarm의 airmode 연동 토글)	PG_PID_PROFILE 11→12

2. 신규 커스텀 CLI 파라미터

2.1 alt_hold_full_low_is_max_descend

항목	값
범위	OFF / ON
기본값	OFF (v1 도입, 기존 스톱 동작 유지)
소속(PG)	PG_ALTHOLD_CONFIG

설명: 스로틀 스틱이 min_check(THROTTLE_LOW) 미만일 때의 동작을 결정합니다. 스톱 Betaflight는 이 구간에서 스틱 위치와 무관하게 항상 강제 호버(고도 유지)로 처리합니다.

OFF(기본): THROTTLE_LOW 구간에서 stickFactor를 계산하지 않고 기본값 0.0(강제 호버)으로 둬 — 스톱과 동일.

ON: THROTTLE_LOW 구간을 '최대 하강 커맨드'(stickFactor = -1.0, 고정값)로 해석 — 스틱을 최하단까지 내리면 즉시 최대 하강률로 하강 시작.

실제 하강 강도는 이 토글이 아니라 alt_hold_climb_rate가 결정합니다 (3절 참고) — ON으로 켜도 climb_rate를 낮춰두면 하강 속도 자체는 완만합니다.

2.2 alt_hold_deadband_low

항목	값
범위	0 ~ 70 (%)
기본값	20
소속(PG)	PG_ALTHOLD_CONFIG

설명: 하강(저throttle)측 데드밴드 폭을 상승측과 별개로 설정합니다. 기존 공식 alt_hold_deadband는 상승/하강 양쪽에 대칭 적용되었는데, 이 값이 하강측만 독립적으로 제어하도록 분리합니다.

동작 원리: 호버 스로틀을 기준으로 하강측 임계값(lowThreshold)을 결정 — 값이 클수록 스틱을 더 많이 내려야 하강이 시작되고(고정 호버 구간이 넓어짐), 값이 작을수록 미세한 스틱 하강에도 바로 반응합니다. 0%는 데드밴드 자체가 사라져 스틱 노이즈에도 하강 반응이 시작되므로 비추천, 30% 이상은 비례제어 구간이 압축되어 오히려 거칠어질 수 있어 비추천 — 기본값(20%) 근처($\pm 5\%$) 권장.

2.3 alt_hold_hover_throttle

항목	값
범위	0, 또는 1100 ~ 1700
기본값	0 (= ap_hover_throttle 상속)
소속(PG)	PG_ALTHOLD_CONFIG

설명: Alt Hold/Position Hold 전용 호버 스로틀 기준값입니다. 기존에는 GPS Rescue와 Alt Hold/Position Hold가 ap_hover_throttle 하나를 공유했는데, 이 값을 통해 Alt Hold/Position Hold만 독립적인 기준값을 가질 수 있습니다.

0(기본값): ap_hover_throttle을 그대로 상속 — 동작 변화 없음, 기존과 100% 동일.

1100~1700: 그 순간부터 Alt Hold/Position Hold가 GPS Rescue와 다른 자기만의 호버 기준값 사용. 이 값은 alt_hold_deadband/alt_hold_deadband_low 데드밴드 계산의 중심점으로도 함께 쓰입니다.

GPS Rescue는 이 설정과 무관하게 항상 ap_hover_throttle을 그대로 사용합니다(동작 변화 없음).

2.4 landing_disarm_airmode_off_only (v3 신규)

항목	값
범위	OFF / ON
기본값	OFF (공식과 100% 동일 동작)
소속(PG)	PG_PID_PROFILE (프로파일별 설정 가능)

설명: 기존 공식 파라미터 landing_disarm_threshold(EZ Disarm, 착지 충격 감지 자동 디스암)의 발동 조건에 airmode 상태를 추가합니다.

OFF(기본): airmode 상태와 무관하게 작동 — 공식 Betaflight와 동일.

ON: airmode가 켜져 있으면 EZ Disarm 자체가 호출되지 않음(완전 비활성). airmode를 꺼야만 정상 작동합니다.

```
// pid.c pidController() 내부 (v3 패치)
if (pidRuntime.useEzDisarm &&
    (!pidRuntime.landingDisarmAirmodeOffOnly || !isAirmodeEnabled())) {
    disarmOnImpact();
}
```

사용 목적: AUX 스위치로 airmode를 켜고 끄며 운용하는 조종자(평상시 비행=ON, 착륙 시=OFF)의 경우, 이 값을 ON으로 해두면 평상시 비행 중엔 EZ Disarm이 완전히 꺼져 오작동 디스암 위험이 없고, 착륙을 위해 airmode를 끄는 순간부터만 EZ Disarm이 작동합니다.

주의(확인된 부수효과): 파일세이프/GPS Rescue 발동 시, airmode 스위치가 배경된 AUX 채널이 rxfail 설정으로 airmode ON 범위 값에 강제 고정되는 경우(보유 기체 다수 해당) 이 구간에서도 EZ Disarm이 자동으로 비활성됩니다. 자동복귀 중 거친 기동으로 인한 오작동 디스암을 방지하는 효과가 있어 의도적으로 허용하는 경우에만 ON으로 사용하는 것을 권장합니다.

3. 관련 기준(공식) CLI 파라미터

위 커스텀 파라미터들은 아래 기준 공식 Betaflight 파라미터와 함께 동작하거나 상호작용합니다. 커스텀 값을 조정하기 전에 아래 항목들의 현재 설정을 먼저 확인하는 것을 권장합니다.

3.1 alt_hold_deadband / alt_hold_climb_rate

파라미터	범위	기본값	설명
alt_hold_deadband	0~70 (%)	20	상승(고throttle)측 데드밴드 폭. 커스텀 alt_hold_deadband_low와 쌍을 이룸
alt_hold_climb_rate	0~200	50 (=5m/s)	최대 상승/하강 속도. 실질적인 안전도를 결정하는 핵심 변수 — 값×10 = cm/s

커스텀 파라미터의 ON/OFF 자체보다 이 climb_rate 값이 실제 위험도를 좌우합니다. 기본 데드밴드 구간 공식상 THROTTLE_LOW 경계 바로 위에서 이미 stickFactor ≈ -0.87(최대 하강의 87%)에 도달하므로, '안전장치'로 느껴지는 강제호버 구간은 매우 좁습니다. climb_rate를 보수적으로(15~20, 초당 1.5~2m) 낮추는 것이 alt_hold_full_low_is_max_descend의 ON/OFF보다 더 실질적인 안전 대책입니다.

3.2 ap_hover_throttle

GPS Rescue 및 (alt_hold_hover_throttle=0일 때) Alt Hold/Position Hold가 함께 참조하는 기준 호버 스로틀 값입니다. 기체마다 실측 호버 스로틀에 맞춰 CLI에 개별 설정되어 있습니다(예: Mark4_6in 1320).

alt_hold_hover_throttle을 0이 아닌 값으로 바꾸면 Alt Hold/Position Hold만 이 값과 다른 기준을 갖게 되니, 두 값을 다르게 가져갈 명확한 이유가 있을 때만 분리 사용을 권장합니다.

3.3 min_check / max_check

파라미터	역할	커스텀 패치와의 관계
min_check	이 값 미만이면 THROTTLE_LOW로 분류 (아밍 안전, alt_hold_full_low_is_max_descend 등의 경계)	alt_hold_full_low_is_max_descend가 처리하는 구간의 하단 경계
max_check	이 값 이상이면 THROTTLE_HIGH로 분류 (아밍 안전 판정용). 실제 스틱 값을 클리핑하지 않음	PWM_RANGE_MAX(2000, 고정 상수)와는 별개 — 수신기가 실제 2000까지 출력하면 max_check 설정과 무관하게 stickFactor=1.0 완전 상승 가능

3.4 landing_disarm_threshold (EZ Disarm)

항목	값
범위	0~250 (0=비활성)
기본값	0 (권장 활성화값은 대략 100 전후, 코드 주석 기준)
내부 스케일	landingDisarmThreshold = 설정값 × 10.0, acc.jerkMagnitude(g/s 단위)와 직접 비교

착지 시 가속도 변화율(저크)이 이 임계값을 넘으면 자동으로 disarm합니다. 발동 조건은 다음과 같습니다 (pid.c disarmOnImpact()):

```
if (wasThrottleRaised()           // 이륙 이력(래치)
    && ((getMaxRcDeflectionAbs() < 0.05f // 스틱 중앙 ±5%
        && mixerGetRcThrottle() < 0.05f // 스로틀 5% 미만
        || FLIGHT_MODE(ALT_HOLD_MODE))) { // 또는 Alt Hold 모드 중
    // Alt Hold 중 착지고도 이하이면 민감도 1.5배
    if (jerkMagnitude * sensitivity > landingDisarmThreshold) {
        disarm(DISARM_REASON_LANDING);
    }
}
```

v3의 landing_disarm_airmode_off_only는 이 함수를 호출할지 말지만 결정하며, 함수 내부 로직은 전혀 수정하지 않습니다.

3.5 mixer_type = EZLANDING 및 ez_landing_* 계열

파라미터	범위	기본값	설명
mixer_type	LEGACY/EZLANDING 등	-	EZLANDING이면 착지 판단 시 모터 최대 출력을 강제로 제한
ez_landing_threshold	0~200	25	이 값 이하 스틱 편차일 때 착지로 간주 시작
ez_landing_limit	0~75 (%)	15	착지로 판단됐을 때 모터 최대 출력 상한
ez_landing_speed	0~250	50	이 속도 이하일 때 착지로 간주(GPS 속도 기준)

보유 기체 전체 현재 mixer_type = EZLANDING 아님으로 변경 완료(사용자 조치). Alt Hold/Position Hold는 평상시에도 RPY 스틱이 중앙에 가까워 '착지 중' 오판 위험이 있고, alt_hold_full_low_is_max_descend = ON으로 급하강 후 PID가 급제동을 요구하는 순간 GPS 속도가 0에 가까워지며 스틱도 중앙이라 EZLANDING이 착지로 오판 → 제동에 필요한 추력 상한을 오히려 낮춰버릴 위험이 있었기 때문입니다.

3.6 feature AIRMODE / aux BOXAIRMODE

airmode 상태는 isAirmodeEnabled() = featureIsEnabled(FEATURE_AIRMODE) || IS_RC_MODE_ACTIVE(BOXAIRMODE)로 결정됩니다. 보유 기체 전체는 CLI에서 feature -AIRMODE(피쳐 자체 OFF)로 설정되어 있고, AUX 스위치에 BOXAIRMODE(permanentId 28)가 배정되어 있어 isAirmodeEnabled()는 곧 AUX 스위치 상태 그 자체입니다. 이 덕분에 landing_disarm_airmode_off_only가 사용자의 실제 조작 습관과 정확히 맞물립니다.

페일세이프/GPS Rescue 시에는 해당 AUX 채널이 rxfail 설정값(대부분 airmode ON 범위 내)으로 강제 고정되어, 신호 두절 중에는 항상 airmode = ON으로 처리됩니다(자동복귀 중 충분한 제어권한 확보 목적, Betaflight 표준 설계). landing_disarm_airmode_off_only=ON 사용 시 이 구간에서 EZ Disarm이 함께 비활성됨을 인지하고 있어야 합니다.

4. 계산 공식 요약

Alt Hold 스틱 반응 계산 (alt_hold_multicopter.c)

```
lowThreshold = hoverThrottle - deadbandLow * (hoverThrottle - 1000)
highThreshold = hoverThrottle + deadband * (2000 - hoverThrottle)

rcThrottle < lowThreshold → stickFactor = scaleRange(rcThrottle, 1000, lowThreshold, -1, 0)
lowThreshold~highThreshold → stickFactor = 0 (진짜 데드밴드, 고도 고정)
rcThrottle > highThreshold → stickFactor = scaleRange(rcThrottle, highThreshold, 2000, 0, 1)

targetVelocity = stickFactor * (climb_rate * 10) // cm/s
```

hoverThrottle은 alt_hold_hover_throttle이 0이 아니면 그 값, 0이면 ap_hover_throttle을 사용합니다. min_check 미만(THROTTLE_LOW) 구간은 이 공식이 아니라 alt_hold_full_low_is_max_descend 값으로 별도 결정됩니다(OFF=0 고정, ON=-1.0 고정).

EZ Disarm 판정 (pid.c)

```
landingDisarmThreshold = landing_disarm_threshold(CLI) * 10.0
jerkMagnitude = |가속도 변화율| (g/s 단위, sensors/acceleration.c)
sensitivity = (ALT_HOLD_MODE 중 && 착지고도 이하) ? 1.5 : 1.0

jerkMagnitude * sensitivity > landingDisarmThreshold → disarm(DISARM_REASON_LANDING)
```

5. 권장 설정 및 CLI 예시

Alt Hold 시험 설정 (보수적 시작값)

```
set alt_hold_climb_rate = 15          # 초당 1.5m로 보수적으로 시작
set alt_hold_deadband_low = 20        # 기본값 유지 권장
set alt_hold_full_low_is_max_descend = OFF  # 1단계: 데드밴드 분리만 우선 검증
set alt_hold_hover_throttle = 0        # 처음엔 상속(0)으로 검증
save
```

EZ Disarm + airmode 연동 설정 (AUX 스위치로 airmode on/off 운용 시)

```
set landing_disarm_threshold = 50      # 착지 충격 판정 임계값(민감도)
set landing_disarm_airmode_off_only = ON  # airmode OFF일 때만 EZ Disarm 작동
save
```

landing_disarm_threshold의 코드 주석은 "relatively safe values are around 100"이라고 안내합니다. 50은 상대적으로 민감한 값이므로, 처음엔 70~80 정도로 시작해 블랙박스로 실제 착지 충격 크기를 확인한 뒤 필요시 낮추는 것을 권장합니다.

6. 안전 점검 요약

점검 항목	결과
아밍 전 ALTHOLD/POSHOLD 스위치 ON 상태 → 아밍 차단	기존 표준 기능, 패치와 무관. 스위치 OFF로 아밍 후 이륙 후 ON 권장
wasThrottleRaised() 래치로 인한 모드 유지	문제 없음 — 이륙 후 스로틀이 낮아져도 모드가 임의로 꺼지지 않음
EZ Disarm과 Position Hold의 관계	다수 기체에서 ALTHOLD/POSHOLD가 같은 AUX 스위치에 배정되어 있어 Position Hold 활성화 시 Alt Hold도 함께 활성화됨 — 패치 로직이 Position Hold 중에도 동작
mixer_type = EZLANDING과의 상호작용	위험 확인되어 보유 기체 전체 EZLANDING 비활성화 완료(사용자 조치)

점검 항목	결과
EZ Disarm ↔ airmode 상시 연동 위험	v3의 landing_disarm_airmode_off_only로 해결 — 평상시 비행 중 오작동 방지, 착륙 시에만 작동하도록 분리 가능
페일세이프/GPS Rescue 중 EZ Disarm 상태	landing_disarm_airmode_off_only=ON 시 이 구간에서도 비활성(코드 검증 완료) — 자동복귀 중 오작동 방지 목적으로 사용자가 의도적으로 수용

7. 기체별 빌드/펌웨어 매핑

기체	보드	MCU	비고
MARIO5	SPEEDYBEEF405V4	STM32F405	
Mark4_6in	JHEF405PRO	STM32F405	MARIO5와 hex 동일(MD5 일치)
AOS_UL7_O4	SPEEDYBEEF405V4	STM32F405	MARIO5와 hex 동일
TJRC_10	MATEKF722SE (듀얼자이로)	STM32F7X2	
8IN-KOPIS_X8	SPEEDYBEEF7V3	STM32F7X2	OCTOX8 믹서
CHIMERA7	FLYWOOOF722PROV2	STM32F7X2	
AOS_UL7_X8	MATEKF722HD	STM32F7X2	OCTOX8 믹서
Explorer LR4	JHEF7DUAL (듀얼자이로)	STM32F7X2	EZ Disarm 활성화 기체 — airmode_off_only 적용 권장 1순위
X8_5INCH	MATEKH743	STM32H743	OCTOX8 믹서, Cortex-M7

같은 MCU를 쓰는 기체끼리는 컴파일된 hex 바이너리가 100% 동일하지만, 제조사별로 보드 리소스 매핑(핀 배선)이 완전히 다르므로 플래시 후 반드시 각 기체 자신의 DIFF ALL 전체를 재적용해야 합니다. hex는 "핀 매핑 능력"만 제공하고, 실제 핀 역할은 CLI 설정이 결정하는 Unified Target 구조이기 때문입니다.

본 문서는 커스텀 빌드 사용을 위한 참고 자료이며, 실비행 적용은 프롭 제거 벤치 테스트 → 개발지 저고도 단계적 비행 시험을 거쳐 사용자 자체 판단과 책임 하에 진행합니다.

8. 부록: 스로틀 스틱 진입 래치 (Entry Stick Latch) — 2025.12.5 v5

2025.12.5 커스텀 빌드(브랜치 custom-patch/alt-hold-throttle-range, 보드별 slim 빌드 v5)에 새로 추가된 기능입니다. CLI 파라미터는 아님(코드에 고정, PG 버전 변경 없음) — 이 문서의 2절 4종 CLI 파라미터와는 별개로 동작합니다.

8.1 배경 및 문제

Alt Hold/Position Hold에 진입하는 순간 조종자의 스로틀 스틱이 호버 위치에서 살짝 벗어나 있으면, 기존 로직은 그 스틱 위치를 곧바로 alt_hold_deadband/alt_hold_deadband_low 기준과 비교해 즉시 상승·하강 커맨드로 해석할 수 있습니다. 즉, 모드 진입 직후 스틱이 정확히 중앙(호버)에 있지 않으면 조종자가 의도하지 않은 고도 변화가 시작될 수 있습니다.

8.2 동작

Alt Hold 진입 순간(altHoldProcessTransitions에서 비활성→활성 전환 시점)의 rcCommand[THROTTLE] 값을 entryThrottle로 저장하고, entryLatched = true로 래치를 건다. 래치가 걸려 있는 동안은 스틱 위치와 무관하게 고도를 그대로 유지하며(stickFactor = 0), alt_hold_full_low_is_max_descend도 이 구간에서는 적용되지 않는다. 스틱이 진입 시점 값에서 PWM 기준 5%(1000~2000 범위의 5% = 50, 예: 진입 시 1500이면 1450 이하 또는 1550 이상) 이상 움직이면 래치가 풀리고, 그 이후부터는 기존 커스텀 로직(alt_hold_hover_throttle, alt_hold_deadband, alt_hold_deadband_low, alt_hold_full_low_is_max_descend)이 호버 스로틀 대비 스틱의 '절대 위치' 기준으로 그대로 적용된다. 한 번 풀린 래치는 Alt Hold를 꺾다가 다시 켜기 전까지 재적용되지 않는다. 페일세이프/GPS Rescue 하강 오버라이드는 래치 상태와 무관하게 그대로 최우선 적용된다(failsafelsActive() 분기가 래치 로직 이후에 무조건 stickFactor를 덮어씀).

8.3 구현 (src/main/flight/alt_hold_multirotor.c)

```
#define ALT_HOLD_ENTRY_LATCH_RELEASE_PWM (0.05f * (PWM_RANGE_MAX - PWM_RANGE_MIN))

// altHoldProcessTransitions(): 비활성→활성 전환 시
altHold.entryThrottle = rcCommand[THROTTLE];
altHold.entryLatched = true;

// altHoldUpdateTargetAltitude(): 매 틱
if (altHold.entryLatched &&
    fabsf(rcCommand[THROTTLE] - altHold.entryThrottle) >= ALT_HOLD_ENTRY_LATCH_RELEASE_PWM) {
    altHold.entryLatched = false;
}
if (altHold.allowStickAdjustment && !altHold.entryLatched) {
    // 기존 stickFactor 계산 (데드밴드/최대하강 포함)
}
```

8.4 원본 출처 및 이식

jsungho/betaflight2026.6.x_custom 저장소 브랜치 custom-patch/alt-hold-throttle-range-2026.6.2, 커밋 665f62a9e ("Alt Hold: hold altitude on entry until throttle stick moves 5%")의 src/main/flight/alt_hold_multirotor.c 변경을 이식했습니다. 2025.12.5는 alt_hold.h/alt_hold_multirotor.c 코드 구조(변수명·함수명)가 2026.6.2와 달라(예: autopilotCaptureHoverThrottleForAltHold() 등 2026.6.2 전용 함수 부재) 동일한 altHoldState_t 구조체에 entryLatched/entryThrottle 필드를 추가하고, 이 저장소의 altHoldProcessTransitions / altHoldUpdateTargetAltitude 함수에 같은 동작을 재현하는 방식으로 이식했습니다.

8.5 영향받는 파라미터

alt_hold_hover_throttle, alt_hold_deadband, alt_hold_deadband_low, alt_hold_full_low_is_max_descend(2절 참고) — 이 값들은 래치가 풀린 이후부터 적용되며, 값 자체나 기본값은 변경되지 않았습니다. Position Hold의 좌우/전후 스틱 로직은 이번 패치의 영향을 받지 않습니다.

8.6 검증

전 기체(10대) 보드별 slim 재빌드 성공, 컴파일 경고/오류 없음, 플래시 오버플로 없음. 4종 커스텀 CLI 파라미터 문자열(alt_hold_deadband_low / alt_hold_full_low_is_max_descend / alt_hold_hover_throttle / landing_disarm_airspeed_off_only) 및 자력계 관련 문자열이 hex 바이너리에 그대로 포함됨을 확인. #pragma message 진단 삽입(임시, 검증 후 원복)으로 ALT_HOLD_ENTRY_LATCH_RELEASE_PWM이 실제로 전처리 단계에서 정의됨을 확인. 파일명 접미사는 custom_v4_slim → custom_v5_slim으로 갱신.

8.7 권장 시험 절차

① 프롭 제거 벤치에서 Alt Hold 진입 시 스로틀 스틱을 진입 위치 근처에서 작게($\pm 5\%$ 미만) 흔들어도 고도 목표가 변하지 않는지(로그상 targetAltitudeCm 고정) 확인. ② 스틱을 5% 이상 움직였을 때만 상승/하강 커맨드가 반응하기 시작하는지 확인. ③ 한 번 래치가 풀린 뒤 스틱을 다시 진입 위치로 돌려도 래치가 재적용되지 않는지(모드를 껐다 켜야만 재래치) 확인. ④ 블랙박스 rcCommand[THROTTLE]과 목표고도 궤적을 함께 로깅해 래치 해제 시점을 사후 검증.

9. 부록: Alt Hold 해제 대기(Exit Hold) + OSD "ALT WAIT" 표시 — 2025.12.5 v6

2025.12.5 커스텀 빌드(브랜치 custom-patch/alt-hold-throttle-range, 보드별 slim 빌드 v6)에 새로 추가된 기능입니다. CLI 파라미터는 아님(코드에 고정, PG 버전 변경 없음). 8장의 진입 스틱 래치(v5)와는 별개 기능이며, 둘은 함께 동작합니다.

9.1 배경

기존에는 Alt Hold 스위치를 끄는 순간 즉시 모드가 해제되어, 그 순간 스로틀 스틱이 호버 위치와 다르면 추력이 갑자기 바뀔 수 있었습니다.

9.2 동작

Alt Hold 스위치를 끈 순간에 Alt Hold가 활성 상태였다면 즉시 해제하지 않고 '해제 대기' 상태로 전환해 고도를 계속 유지합니다. 대기 중에는 스틱에 의한 고도 조절을 완전히 무시합니다. 스로틀 스틱이 $ap_hover_throttle \pm 5\%$ (PWM 50, 1000~2000 범위) 안에 들어오면 그 순간 Alt Hold를 완전히 해제합니다. 기준은 $ap_hover_throttle$ 만 사용하며, $alt_hold_hover_throttle$ 이나 thr_mid 는 기준이 아닙니다. $ap_hover_throttle$ 이 0이면 Alt Hold의 실호 호버 값으로 대체합니다($alt_hold_hover_throttle$ 이 설정돼 있으면 그 값, 아니면 $ap_hover_throttle$ 상속분과 동일). 스위치를 끈 순간 스틱이 이미 그 구간 안에 있으면 대기 없이 바로 해제됩니다. 대기 중 스위치를 다시 켜면 대기를 취소하고 일반 Alt Hold로 복귀하며, 이때 진입 스틱 래치(8장)가 현재 스틱 위치로 새로 걸립니다. 파일세이프/GPS Rescue 하강 오버라이드는 대기 상태와 무관하게 항상 최우선 적용됩니다.

9.3 OSD 표시

OSD 비행모드 요소(osdElementFlymode)에서 해제 대기 중이면 "ALT WAIT"를 경고색(DISPLAYPORT_SEVERITY_WARNING)으로 표시합니다. 표시 우선순위: !FS!(파일세이프) → RESC(GPS Rescue) → HEAD(헤드프리) → PASS(패스스루) → ALT WAIT → POSH(Position Hold) → ALTH(Alt Hold).

9.4 구현

```
// alt_hold_multirotor.c
#define ALT_HOLD_EXIT_HOVER_BAND_PWM (0.05f * (PWM_RANGE_MAX - PWM_RANGE_MIN))

bool altHoldRequestActive(bool switch0n)
{
    const bool wasSwitch0n = altHold.prevSwitch0n;
    altHold.prevSwitch0n = switch0n;
    if (switch0n) {
        if (altHold.exitPending) {
            altHold.exitPending = false;
            altHold.entryThrottle = rcCommand[THROTTLE];
            altHold.entryLatched = true;
        }
        return true;
    }
    if (wasSwitch0n && altHold.isActive) {
        altHold.exitPending = true;
    }
    if (altHold.exitPending) {
        const uint16_t apHover = autopilotConfig()->hoverThrottle;
        const float hoverPwm = apHover != 0 ? (float)apHover : altHold.hoverThrottle;
        if (fabsf(rcCommand[THROTTLE] - hoverPwm) <= ALT_HOLD_EXIT_HOVER_BAND_PWM) {
            altHold.exitPending = false;
        }
    }
}
return altHold.exitPending;
```

```

}

// fc/core.c: IS_RC_MODE_ACTIVE(BOXALTHOLD) -> ALT_HOLD_SWITCH_REQUEST()
// (!USE_WING: altHoldRequestActive(...), USE_WING: 기존 그대로)
// DISABLE_FLIGHT_MODE(ALT_HOLD_MODE) 분기에서 altHoldClearExitPending() 호출

// osd/osd_elements.c: osdElementFlymode()에 ALT WAIT 분기 추가
// (PASS 다음, POSH 이전)

```

9.5 원본 출처 및 이식

jsungho/betaflight2026.6.x_custom 저장소 브랜치 custom-patch/alt-hold-throttle-range-2026.6.2, 커밋 b83985b("Alt Hold: keep holding after switch-off until throttle reaches hover +/-5%, OSD ALT WAIT") 및 커밋 e4593eb("Alt Hold exit hold: use ap_hover_throttle as reference")의 src/main/flight/alt_hold_multitrotor.{c,h}, src/main/fc/core.c, src/main/osd/osd_elements.c 변경을 이식했습니다. 진입 스틱 래치 커밋(665f62a, 8장)은 이번 이식 대상이 아닙니다. 2026.6.2는 autopilotGetEffectiveHoverThrottlePwm()(2026.6.2 전용 함수)을 폴백으로 썼지만 2025.12.5에는 이 함수가 없어, 이 저장소가 v2 패치에서 이미 계산해 쓰는 altHold.hoverThrottle(alt_hold_hover_throttle 이 설정돼 있으면 그 값, 아니면 ap_hover_throttle 상속)을 동일한 역할의 폴백으로 사용했습니다. fc/core.c는 2026.6.2의 AUTOPILOT_MODE/flightPlanNavIsRescueDescentActive() 조건이 2025.12.5에는 없어(해당 기능 자체 없음) 제외하고, IS_RC_MODE_ACTIVE(BOXALTHOLD) -> ALT_HOLD_SWITCH_REQUEST() 치환만 반영했습니다.

9.6 검증

전 기체(10대) 보드별 slim 재빌드 성공, 컴파일 경고/오류 없음, 플래시 오버플로 없음(24.35~78.90% 사용). 4중 커스텀 CLI 파라미터, 자력계 관련 문자열, "ALT WAIT" 문자열이 전 기체 hex 바이너리에 포함됨을 확인. #pragma message 진단 삽입(임시, 검증 후 원복)으로 ALT_HOLD_EXIT_HOVER_BAND_PWM이 실제로 전처리기 단계에서 정의됨을 확인. 파일명 접미사는 custom_v5_slim -> custom_v6_slim으로 갱신.

9.7 권장 시험 절차

① 프롭 제거 벤치에서 Alt Hold 중 스위치를 끄되 스틱이 호버 위치에서 벗어나 있는 상태로 시험 — 스위치를 꺼도 고도가 유지되고(targetAltitudeCm 고정), OSD에 ALT WAIT가 뜨는지 확인. ② 스틱을 ap_hover_throttle $\pm 5\%$ 안으로 가져왔을 때 그 순간 해제되고 OSD 표시가 사라지는지 확인. ③ 대기 중 스위치를 다시 켰을 때 정상적으로 Alt Hold로 복귀하고(진입 스틱 래치가 새로 걸림) 대기 상태가 취소되는지 확인. ④ 스위치를 끄는 순간 스틱이 이미 호버 근처였을 때 대기 없이 즉시 해제되는지 확인. ⑤ 블랙박스로 rcCommand[THROTTLE], ap_hover_throttle, 목표고도, ALT WAIT 표시 구간을 함께 로깅해 해제 타이밍을 사후 검증.

10. 부록: 해제 대기 — 호버 구간 건너뛸 결함 수정 + 시뮬레이션 테스트 — 2025.12.5 v7

9장(해제 대기, v6)에 이어지는 수정이다. CLI 파라미터는 아님(코드에 고정, PG 버전 변경 없음).

10.1 문제

v6의 해제 대기는 매 사이클(100Hz, 10ms 간격) 스로틀 스틱이 $ap_hover_throttle \pm 5\%$ (PWM 50) 구간 안에 '머무르는지'만 검사했다. 조종자가 스틱을 매우 빠르게 움직이면 한 사이클과 다음 사이클 사이에 스틱 값이 이 구간을 통째로 건너뛸 수 있다 — 예를 들어 한 틱에는 구간보다 한참 위(예: 1600), 다음 틱에는 구간보다 한참 아래(예: 1000)로 가는 경우, 구간 안에 정확히 머무른 샘플이 한 번도 없어 Alt Hold가 영원히 해제되지 않는 결함이 있었다.

10.2 수정

해제 대기가 시작되는 순간과 매 사이클마다 직전 스로틀 값(exitPrevThrottle)을 저장해두고, (스틱 - ap_hover_throttle)의 부호가 직전 샘플과 이번 샘플 사이에서 바뀌었으면(=호버 값을 가로질렀으면) 구간 안에 정확히 들어오지 않았어도 그 자리에서 해제한다. 위쪽에서 아래로, 아래쪽에서 위로 넘어가는 경우 모두 해당한다. 해제 조건 = $|스틱 - ap_hover_throttle| \leq 50$ 이거나, 직전 사이클 대비 호버 값을 가로지른 경우.

```
// alt_hold_multirotor.c
typedef struct {
    ...
    float exitPrevThrottle; // v7: 직전 스로틀 샘플(호버 가로지름 판정용)
} altHoldState_t;

bool altHoldRequestActive(bool switchOn)
{
    ...
    if (wasSwitchOn && altHold.isActive) {
        altHold.exitPending = true;
        altHold.exitPrevThrottle = rcCommand[THROTTLE];
    }
    if (altHold.exitPending) {
        const uint16_t apHover = autopilotConfig()->hoverThrottle;
        const float hoverPwm = apHover != 0 ? (float)apHover : altHold.hoverThrottle;
        const float prevDelta = altHold.exitPrevThrottle - hoverPwm;
        const float delta = rcCommand[THROTTLE] - hoverPwm;
        altHold.exitPrevThrottle = rcCommand[THROTTLE];
        if (fabsf(delta) <= ALT_HOLD_EXIT_HOVER_BAND_PWM
            || (prevDelta < 0.0f) != (delta < 0.0f)) {
            altHold.exitPending = false;
        }
    }
    return altHold.exitPending;
}
```

10.3 해제 대기 중 자세/스로틀 동작 (설계 확인)

해제 대기 중에도 ALT_HOLD_MODE 자체는 계속 켜진 상태다. 즉 자세 제어는 앵글(자동 수평) 모드 그대로 유지되고, 대기 중 스로틀 스틱을 아무리 내려도 고도가 내려가지 않는다(목표 고도를 그대로 유지). 이는 설계된 동작이다 — 스틱이 호버 구간에 들어오거나 호버 값을 가로지르기 전까지는 스틱 입력을 의도적으로 완전히 무시한다. 처음 접하면 '스로틀을 내렸는데 안 떨어진다'고 당황할 수 있으므로 비행 전 반드시 숙지한다.

10.4 시뮬레이션 테스트

jsungho/betaflight2026.6.x_custom 저장소 브랜치 custom-patch/alt-hold-throttle-range-2026.6.2, 커밋 415e8c4("Alt Hold exit hold: release when stick crosses hover (fast flick); sim tests")의 AltholdCustomSim 테스트 스위트(해제 대기 관련 부분)를 src/test/unit/althold_unittest.cc에 이식했다. 2025.12.5는 2026.6.2 전용

API(`autopilotGetEffectiveHoverThrottlePwm` 등)가 없어 이 저장소의 실제 `pg/autopilot.h`, `pg/alt_hold.h` 구조(`autopilotConfigMutable`, `altHoldConfigMutable`, `rxConfigMutable()`→`mincheck` 등)에 맞춰 스텝을 다시 구성했다. 호스트(PC, x86) 빌드라 실제 타깃 빌드와 무관하게 `cd src/test && make test_althold_unittest` 로 바로 실행할 수 있다.

검증한 13개 시나리오(전부 통과): 진입 스틱 래치가 고도를 유지/해제하는 경우(3개, v5), 해제 대기가 호버 구간 밖에서는 고도를 유지하다가 `ap_hover_throttle ±5%` 진입 시 해제, 기준이 `alt_hold_hover_throttle`이 아니라 `ap_hover_throttle`임을 구분, 스위치를 끈 순간 스틱이 이미 구간 안이면 즉시 해제, 빠른 스틱 이동으로 구간을 위→아래/아래→위로 건너뛰어도 해제(v7 수정 대상), 대기 중 스위치 재투입 시 재래치, 디스암 시 대기 삭제, Alt Hold에 진입한 적이 없는 상태에서 스위치를 끄면 대기가 시작되지 않는 경우.

10.5 빌드/바이너리 검증

전 기체(10대) 보드별 slim 재빌드 성공(파일명 접미사 `custom_v6_slim` → `custom_v7_slim`), 컴파일 경고/오류 없음, 플래시 오버플로 없음(24.35~78.92% 사용, v6 대비 사실상 동일 — float 필드 1개와 분기 하나 추가라 증가분 미미). 4종 커스텀 CLI 파라미터와 "ALT WAIT" 문자열이 전 기체 hex 바이너리에 포함됨을 확인. `#pragma message` 진단 삽입(임시, 검증 후 원복)으로 `ALT_HOLD_EXIT_HOVER_BAND_PWM`이 v7 코드에서도 전처리기 단계에서 실제로 정의됨을 확인.

10.6 권장 시험 절차 (9.7절에 추가)

① 프롭 제거 벤치에서 Alt Hold 해제 대기 중 스틱을 천천히 호버 근처로 가져와 정상 해제되는지 확인(9.7절 기존 절차). ② 해제 대기 중 스틱을 한쪽 끝에서 반대쪽 끝으로 매우 빠르게(플릭) 움직여, 호버 구간에 머무는 시간 없이도 그 순간 해제되는지 위/아래 양방향 모두 확인 — 이것이 v7에서 새로 고친 부분이다. ③ 해제 대기 중 스로틀을 완전히 내려도 고도가 유지되고 하강하지 않는지(설계대로) 확인. ④ 블랙박스 `rcCommand[THROTTLE]`, `ap_hover_throttle`, 목표고도, ALT WAIT 표시 구간을 함께 로깅해 실제 가로지름 판정 타이밍을 사후 검증.